Judged Whole

One Gate for Untrusted Verification Infrastructure

ANONYMOUS AUTHOR(S)

Verification infrastructure written by a language model cannot be trusted by construction, and neither can a proof it emits. We describe a system in which that is not a problem to be solved but the premise of the design. Every executable — interpreters, translators between languages, searches for evidence, certificate checkers, the maps that carry evidence home — is generated text, and none of it is believed: each enters through one gate that runs it against its own shipped cases and its own shipped mutants, and each result is graded by *where along its route the evidence was judged.* A witness is judged by replaying it through the interpreter of the language the question lives in; a certificate by re-checking it with that language's checker; a claim with nothing behind it is judged by nothing and says so. The grade is a distance — certified at gap zero, checked one or more translations away, claimed when no judgment ever ran — and what a result still rests on, its residual, is computed from the log by removing everything upstream of the last judgment. The trusted base is exactly the admitted judges; the kernel that runs the gate stands outside it and is proved where it is mathematics, falsified where it is operation, and in its pure half doubled by a clean-room second implementation that must agree byte for byte.

We measure the design at one frozen commit, and we measure it against faults nobody shipped. Of 1,672 sampled source mutants of the generated translators, carry-backs, and searches, forced past the gate and played on questions whose answers are known, 16,720 plays produce 133 wrong answers — 127 graded claimed, 6 graded checked, *none* certified — and every wrong answer's residual names the faulty component. Of 700 mutants of the seven judges, the gate refuses half; a differential probe of the survivors finds six real soundness holes, and revisions that add only test vectors, each expected value confirmed by an outside tool that never enters the system, close them. On four benchmarks pinned from HWMCC'24 and SV-COMP 2025 the system settles 48 of 74, 28 of 79, 47 of 80, and 12 of 55 questions, 49 universal answers resting on a home judge alone; the generated searches are not competitive with the solvers of those competitions, and the paper says why performance was not pursued first, draws the competitions' results and a host calibration of their solvers beside each board, and says how the loop that produced the tree is run; the negatives are reported as results. Nothing in the kernel is about programs or machines: a domain is a root language with its anchors, it enters by one translator to a hub where searches live, and every search there then reaches it — the software questions are answered by hardware searches today, and domains of mathematics or the natural sciences, with their solvers as searches, would enter the same way.

1 INTRODUCTION

Verification infrastructure is now being written by language models: interpreters, translators, invariant generators, proof scripts, and the checkers that are supposed to catch the rest, each text produced by a process that cannot be inspected and is wrong at an unknown rate. Two answers are current. One verifies the artifact: repair the proof until a checker accepts it, or generate the code together with a specification and a proof [15, 37]. The other distrusts the artifact and checks its outputs: certifying algorithms [26], proof-carrying code [30], validated verification witnesses [8]. Both are about one artifact at a time. Neither says what happens when the checker is generated too, when the translation that carried the program to the checker is generated, when the map that carries the verdict back is generated, and when the search that produced the evidence was generated last week by the same process.

This paper describes a system built on the premise that every one of those pieces is untrusted, and judged whole. It started from an empty repository with a specification, and everything that has entered since — every interpreter, translator, search, checker, and carry-back — is Python written by a language model and admitted through one gate. The separation that organizes it is one sentence: **generation produces syntax; only interpretation produces truth.** A *language*

is a syntax with an *interpreter* and, per declared *certificate form*, a *checker*; these are the *judges*, and the trusted base is exactly the admitted judges, a list the kernel prints. Every other executable is a *transport* — translator, carry-back, or budgeted search — untrusted whatever it computes. A question lives at a language; a *route* carries it across *pairs* of languages to a search; whatever comes back is judged where it lands, and the *gap* of a result is the number of hops between the question and the last judgment its evidence passed (Fig. 2). Gap zero is *certified*: a witness replayed, or a certificate discharged, by the home judge. A positive gap is *checked*. No judgment is *claimed*. The *residual* — what a result still rests on — is the lineages of the hops inside the gap plus the judge that ran; every judgment removes everything upstream of it (Theorem 4.2, proved in Lean).

A design like this makes two claims that can be wrong. The first is about transports: a faulty translator, carry-back, or search can cost answers but can never forge a *certified* one, and whatever it corrupts says so in its grade and names it in its residual. The second is about judges: the trusted base is small enough, and tested well enough, to be believed. We test both by fault injection (Section 7.1). For the first we mutate every transport in the registry, *remove the gate*, force each mutant in, and play it on questions whose answer is known: 16,720 plays of 1,672 mutants yield 133 wrong answers, 127 *claimed*, 6 *checked* at gap one, none *certified*, every one naming the faulty entry in its residual. For the second we mutate the judges: half of 700 are refused at the gate, the survivors concentrate in the certificate checkers, and a differential probe finds six of them laxer than the checker they mutate — real holes, closed without touching a judge, by revisions that add test vectors whose expected values are confirmed by tools that never enter the system.

The boards are the third measurement (Section 7.3). Two domains have entered — hardware, rooted at BTOR2 [32], and software, rooted at a fragment of C — with RISC-V between them, anchored by no domain. Nothing in the kernel is specific to either: a domain enters by one pair to a hub where searches live, and every search there then reaches its questions, which is how a software question is answered by a hardware search and how a domain of mathematics or physics would enter (Section 9). Four generated pairs connect the three languages, six generated searches live at BTOR2 and C, and four benchmarks pinned from HWMCC'24 and SV-COMP 2025 stand at 48 of 74, 28 of 79, 47 of 80, and 12 of 55 questions settled, 49 universal answers *certified* at gap zero, no contradiction. The generated searches are not competitive with the solvers whose verdicts sit beside the boards; Section 6 says why performance was not pursued first and draws the column the design reserves for it — the competitions' results and a calibration of their solvers on the playing host beside each board — and Section 6 says how the frontier is pushed — the turns that produced the tree are in Table 4.

AI Use. All of the system — the kernel, its tests, the second lineage of its pure half, the Lean development, every entry in the registry — and the instruments of Section 7 — the fault-injection harness, the generators of test-vector expectations — are code generated by language-model agents (Anthropic's Claude Fable, through the Claude Code command-line agent). Most of this paper's text, its tables, and its figures are generated by the same agents from the specification and the measurement records. No human reviewed the code or the text for semantic correctness. The human contribution is the architecture — the separation of judges from transports, the gate, the decision that grades are computed and never asserted — and the choices of what to build, measure, and claim. No number here originates with a model: each is read from logs and stamps the kernel wrote, the boards and the fault-injection counts regenerate from the deposited records by one command each, the proofs are checked by Lean, and the test vectors' expected values were confirmed by tools that are not generative.

Contributions.

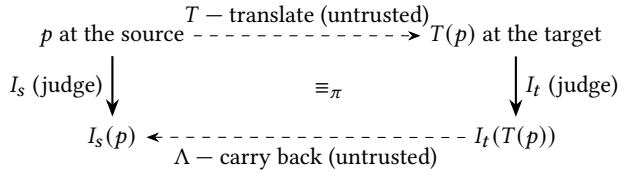


Fig. 1. The square, the judgment on programs. Dashed arrows are transports, generated and untrusted; solid arrows are the two interpreters, the judges. The square is closed empirically, per corpus program, at admission; the observable renaming π is the projection inside it. Where the translation turns one source frame into many target frames, a stimulus map and a bound map ride along and are judged inside the same square against the depths both interpreters report.

- A design in which every executable of a verification system is untrusted generated text and one gate admits all of them: two kinds, three judgments an artifact faces on crossing a pair (the square, replay, discharge), two crossings nothing judges (Section 3).
- Grades as geometry: gap, grade, and residual computed from the log, the order on results and the residual's properties proved (Sections 3.4 and 4); a kernel outside the gate made solid four ways, each worded no stronger than what it verifies (Section 4).
- The loop as an operating procedure — the commands, what an entry ships, what the frontier tells the player, the conjecture order — and the record of its turns (Section 6).
- A fault-injection method for such systems and its results: the no-forge claim tested with the gate removed, the trusted base's exposure measured and localized, six holes found and closed with outside ground truth (Sections 7.1 and 7.2).
- Measurements at one frozen commit, every number tied to a file in the tree, the negatives reported as results (Section 7.3).

What is not claimed. The square is checked empirically, per program, on a pair's corpus; per-program translation correctness is not proved. The seal around generated executables is a blank `PATH` and an emptied environment, measured by tests, not a proof. Half the sampled judge mutants survived admission and a third still do after the revisions; for most of those we can say only that no certificate we could find distinguishes them from the judge. The *corroborated* flag has fired only between two searches, never between two judges. The boards are not ranked against any solver; the competitions' results and one host calibration are recorded beside each question as testimony, and the one pass played against them is a budget move.

2 ONE QUESTION, TWO ROUTES

Suppose a benchmark pins a C program p — a loop over values read from outside, an assertion inside — and asks whether the assertion can fail within twenty steps. In the system that sentence is formal. C is a *language*: a deterministic syntax plus a generated, deterministic interpreter I_C exposing named *observables* — `bad`, whether an assertion failed, and `depth`, how many statements have run — and the certificate forms it can judge, each with a generated checker. The *question* is p, at C, with its *ask*: observable `bad`, mode `exists`, bound 20. The benchmark enters as a *domain*: a *root* language together with its *anchors* — labels, supplied vectors, the recorded testimony of outside tools — the ungenerable half.

No search reasons about C directly; the model-checking searches live at BTOR2, a language of bit-vector machines. A *pair* connects the two: one correspondence written as generated code, a translator T that turns the elaborated control flow of p into a machine, one statement one

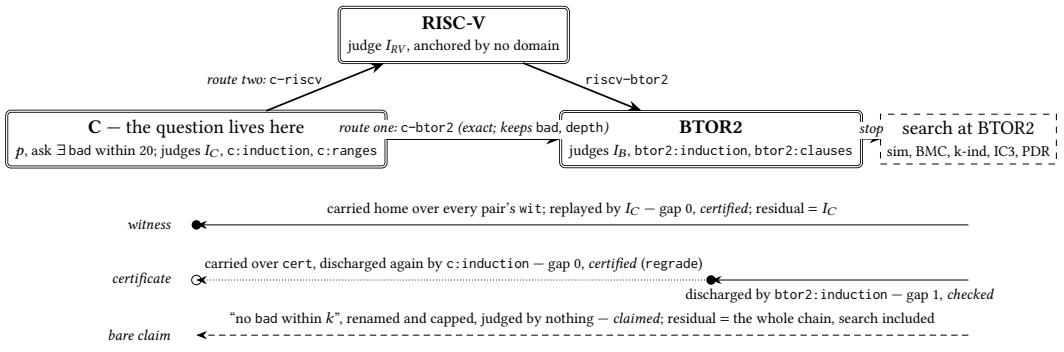


Fig. 2. One question, two routes (Section 2). Above: the question lives at C; route one crosses one pair to the searches at BTOR2, route two crosses two. Below: what a search writes travels back and is judged where it lands — a filled dot is an interpreter or checker run, and the gap counts the hops between the question and the last one. A witness is always replayed at home. A certificate is discharged where it lands — at gap 1 its residual is the lineage of c-btor2 plus BTOR2's judge — and, once the pair carries certificates, again at home by regrade. A bare claim is judged nowhere.

transition, and *carry-backs* Λ for whatever must come home. Three kinds of artifact cross a pair, and each is judged on arrival by an interpreter run. The program crosses forward and is judged by *the square* (Fig. 1), closed for every program of the pair's *corpus* by running both interpreters: $I_s(p) \equiv_\pi \Lambda(I_t(T(p)))$ on the kept observables π. The horizontal arrows are untrusted syntax, the vertical arrows are judges, and nothing horizontal is believed until something vertical has run; a wrong translator loses squares and is refused at the gate. The route ends at a *search*, the one partial transport, budgeted and allowed to return an honest *partial*. What it writes travels back and is judged where it lands (Fig. 2): a *witness* is carried home and replayed by I_C, gap 0, *certified*, and no generated code can corrupt it, since a wrong search, carry-back, or translator all fail the same way, by not producing a stimulus that replays; a *certificate* is discharged by the checker of the language where it lands, *checked* at gap 1 if that is BTOR2, and *certified* once the pair carries it home to C's own checker; a *bare claim* is judged by nothing and floors at *claimed*.

The same question also travels the longer route of Fig. 2, compiled to RISC-V — a language whose interpreter is execution of machine code — then encoded to BTOR2. Pairs compose; a route's forward contract is the componentwise meet of its hops', so a compiler pair that keeps only bad makes the whole route a bad-only route. What a second route can buy is a second *lineage*: results of disjoint generated descent that agree earn the *corroborated* flag — not yet earned by these two, which share one C front end. A *result* is one play of a route on one question, appended to the run's log with value, grade, gap, residual, and cost; cost is recorded and never ranked. Section 3 makes each word exact, Section 4 says how the kernel is made trustworthy, Section 5 lists what was admitted, Section 6 says how the loop is run, and Section 7 reports what fault injection and the plays found.

3 THE DESIGN

The specification is written in a vocabulary of fifty words, listed in its last section, and this section follows it. The registry holds exactly two primitive kinds; everything else — pairs, evidence, grades, routes — is derived, and the kernel computes with the derived forms without storing them as kinds.

3.1 Languages, judges, transports, domains

A **language** is a deterministic syntax plus a deterministic interpreter exposing named observables, plus the certificate forms the language can judge, each shipped with a generated checker. Interpreters and checkers are the **judges** — the system's only semantic devices — and the **trusted base** is exactly the set of admitted judges, each with the anchors and controls that corroborate it. A *root* is a language benchmarks arrive in; its interpreter is corroborated by the domain's anchors and by nothing generated; every other language's interpreter is corroborated empirically by the squares of the pairs that connect it.

A **transport** is a generated function on syntax that declares its *fragment*, the programs it accepts, and refuses outside it. There are three: the *translator* T, source program to target program; the *carry-backs* Λ, a target-side artifact to its source-side form — a witness input, a certificate, an observable name; and the *search*, the one partial transport, given a budget and deterministic for it. A route ends at exactly one search.

A **domain** is a root language with its *anchors* — the two things the loop cannot generate: the format questions arrive in, and the ground truth (labels, supplied vectors, recorded testimony of outside *oracles*) that corroborates the root's interpreter; it owns nothing else and fences nothing, since every admitted language, pair, and search serves every domain.

3.2 Pairs: three judgments, and two crossings nothing judges

A **pair** is the transports that share one correspondence between a source and a target language, declared with a *direction* (exact, over, under), the observables it *keeps*, the artifacts it carries, measured cost, its lineage, and its *corpus*, the programs it ships as its own controls. Three kinds of artifact move across a pair, each judged on arrival by an interpreter run; there is no other kind of check in the system:

artifact	moves	transport	judged on arrival by
a program	forward	`T.py`	**the square**: both interpreters agree on what is kept
a witness	back	`lam_wit.py`	**replay**: the source interpreter runs the program on it
a certificate	back	`lam_cert.py`	**discharge**: the source language's checker re-checks it

Two more things cross a pair, and neither is judged. A **bare claim** — "no failure within bound k" with no certificate — crosses back needing only the pair's observable renaming and its *bound cap*; nothing judges it, which is why it cannot raise trust. A **hint** — seeds, candidates — crosses forward into a search, trust-inert by construction: whatever it seeds is judged on arrival, so a hint can move cost and never a grade. The square is checked empirically, per corpus program, at admission; it is not a primitive and not a proof, and that is the system's founding economy. Direction declares what may cross back — witnesses along exact and under pairs, universal objects along exact and over — and the square is the empirical evidence for the direction claim. A pair that reifies an unrolling declares a bound cap: an unbounded object crossing it arrives as a bound-k fact.

Frames that do not align. A pair whose translation changes the frame granularity — one machine step becoming a run of statements — ships two more transports, judged inside the square: a *stimulus map* (`lam_in.py`) carrying a source stimulus forward, whose inverse on the corpus the witness carry-back must be, and a *bound map* (`lam_bound.py`) carrying an ask forward and a claim back, forward strictly monotone with infinity fixed, back its lower adjoint, both held to the depths the two interpreters report on every corpus stimulus. The player carries bounds through every hop and books a claim that ends below the first source frame as a partial. Certificate forms belong to languages: for a certificate to come home, its form must be declared where it lands, with a checker.

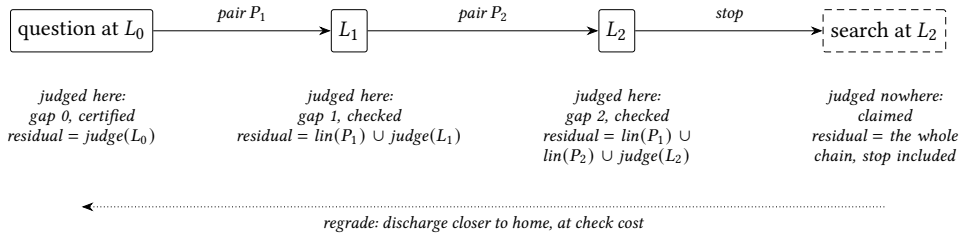


Fig. 3. Grades as geometry. Whatever the search writes is judged where it lands; the gap counts the hops between the question and the last judgment, the residual is the lineage union over the gap segment plus the judge, and at gap 0 nothing but the home judge remains. A bare claim is judged nowhere. Regrade moves a stored certificate down the ladder at check cost.

3.3 Evidence: what a search writes

A search is a transport from a language to evidence about its programs. It declares which observables it can target, it writes, and it pronounces nothing. A **witness** is an input; its judge is the language's own interpreter, by replay, so no generated code stands between the payload and the trust event. A **certificate** is an object of a declared form — a k-induction kernel, a clause invariant, a box per program point — shipped with a checker under the language entry and admitted like everything else: determinism, example certificates that must discharge, mutant certificates that must not. Checkers are judges and meant to be the simplest code in the system; Section 7 measures how far the tree is from that. A **bare claim** is legal and floors at the lowest grade, so a search that writes checkable certificates outgrades one that writes bare claims at the same cost. A **partial** — how far the search got, where it failed — is what a route returns when nothing above came home. A result that decides its question is *settled*: a replayed witness, or a universal claim covering the asked bound.

3.4 Grades are geometry

Trust is computed, per result, from where its evidence was judged (Fig. 3):

gap = the hops between the question and the last judgment the evidence passed
grade = *certified* if the gap is 0; *checked* if it is positive; *claimed* if no judgment ever ran
residual = the lineages of the hops still between the question and that judgment, plus the judge that ran
corroborated = results of disjoint lineage agree — an orthogonal flag

Each judgment removes everything upstream of it from the residual: a certificate discharged at the search's language removes the search — the search may be garbage; the checker validated the object — and discharged where the question lives it removes the route entirely, so *certified* is route-independence (Theorem 4.2). The fail-safe direction is definitional: transport is untrusted, so a wrong or adversarial carry-back can only lose a grade, never forge one. The kernel always replays witnesses where the question lives, chaining carry-backs across the route, so a witness is certified or it is not a result at all; an unreplayable witness is evidence inside a partial. A replayed witness is ground truth, and a covering universal claim beside it is a recorded **contradiction**: the witness stands, and the universal's entire chain is marked falsified. **Lineage** is a declaration about the descent of generated procedures; two judges grown from the same ancestor never corroborate each other.

3.5 Routes, results, the order, the frontier

A **question** is a program, the language it lives in, and its *ask*: an observable, a mode, and a bound. A **benchmark** is a pinned finite set of questions with recorded provenance — a sha256 per program, labels where they exist. A **route** is a sequence of pairs ending at one search; its forward contract is the componentwise meet — fragments intersect, kept observables intersect, bounds cross each hop's bound map and bound caps take the minimum, costs add — and how far an artifact travels back is per kind, as far as every pair on the way carries it, its grade computed from where it was last judged. A **result** is one play of a route on one question, appended to the log:

$$\mathit{result} = \{\ \mathit{question},\ \mathit{route},\ \mathit{budget} : \{\mathit{caps}, \mathit{spent}\},\ \mathit{value},\ \mathit{grade},\ \mathit{gap},\ \mathit{residual},\ \mathit{lineage}\ \}$$

where the value is a replayed witness, a universal claim $\mathit{all}(\mathit{bound} : k \mid \infty,\ \mathit{cert}?)$ graded by its gap, or a partial with a small typed core the kernel orders on — the bound reached — and free-form progress for the player.

Results order per question by one key, stated in the specification because the mechanization and the second lineage of the kernel are both held to it: $(\mathit{level}, \mathit{bound}, \mathit{grade}, \mathit{gap})$, lexicographic, strictly greater is better. Level: 0 a partial, 1 a universal claim below the asked bound, 2 settled. Bound: for a universal claim its bound, ∞ above every number; for a witness, above ∞ itself; for a partial the bound it reached. Grade: ungraded below *claimed* below *checked* below *certified*. Gap: smaller is better, and "no check ever ran" sits below every finite gap. The incumbent survives only while it is strictly better; among equal keys the latest wins. Cost is recorded in every result and never ranked. Two performance moves are trust-free because they are judged: **regrade** re-discharges a stored certificate closer to home at check cost, not search cost, and a shrinking gap is a strictly improving result; **hints** can move minutes of search and not one grade.

The **frontier** of a benchmark is the set of questions whose best result is not settled, each carrying that result and its progress. The registry and the log are append-only, and best-per-question over an append-only log is monotone: the *ratchet* is a property of the data structure. Expanding the frontier means strictly improving some question's best result — level, bound, grade, or gap; never cost. The *board* is a pure function of the log. The **ledger** says what a play bought, in bits — witness surprisal, the log-size of the stimulus space a claim exhausts — profiling recorded beside every result, never ranked, never a grade.

3.6 The generation rule: oracles, not organs

Every implementation is generated, in Python, and lives as committed source inside its registry entry. No wrapped engines, no shelling out, no vendored binaries of someone else's reasoning: nothing that reasons enters except as generated text through the gate. The rule bounds the system, not the evidence about it. At admission time a human or the generator may consult an **oracle** — an existing compiler or solver run entirely outside the sealed runner — and what it says enters the registry the only way ground truth ever enters: as anchors with recorded provenance, corroborating a judge or a transport the way a benchmark label corroborates a root interpreter. An oracle never enters the trusted base, never runs inside a play, and its disagreement with a judge is a dispute to record, never a verdict; the competitions' verdicts and the tools that confirmed the vectors of Section 7.2 are anchors of this kind.

Enforcement is layered and worded no stronger than what each layer verifies: no manifest field can point at a tool; the runner is sealed — every registered executable runs in its own process with an environment emptied to a blank `PATH`, in a scratch directory, under a wall cap, every check run twice with byte-compared output; and every implementation is committed source. **Accelerators**: syntax may accelerate, semantics never does. An entry may ship one accelerator — the same implementation generated again in a faster language — only for the play-time transports T and the

search, admitted solely by byte-agreement with its Python reference on every admission invocation; judges always run the reference.

3.7 The gate

Nothing — no language, pair, search, domain, or any of their implementations, accelerators included — enters the system except by admission, and admission evidence is stamped into the entry by the gate, never self-reported. One discipline for every kind: determinism measured (run twice, byte-compare), the checkable relation run on the entry's own corpus, and two-sided **controls** — the intact implementation must pass and every supplied **mutant** must fail, because a checker that cannot be made to fail is unfalsifiable. Per kind: a language's vectors interpret as expected and each shipped checker discharges its example certificates and refuses its mutant ones; a pair's every carried artifact round-trips per corpus program — squares close, stimuli replay, certificates discharge — with mutants per artifact kind; a search's evidence on its corpus must judge valid and its negative controls fail; a domain names its root and every anchor's provenance. An entry that fails leaves no stamp and no trace in routing.

Revision, not mutation. An admitted entry is never edited. Every **stamp** pins the entry's bytes with a content hash defined in the specification — over every regular file under the entry, excluding the top-level manifest, dotfiles, and bytecode, the map from path to sha256, serialized as JSON with sorted keys, hashed once more — which the loader re-verifies. Extension arrives as a new entry `<name>@<r>` carrying its predecessor's hash. The gate for a revision is the ordinary kind gate plus *conservativity*: the new implementation must byte-agree with its predecessor on the predecessor's whole checkable surface — its vectors or corpus, and for a language the corpora of every admitted pair bound to it. A name binds to its highest admitted revision; predecessors stay in the tree; results name the revision they ran. Adding a carry-back to an admitted pair is the intended common case: the old transports are the conserved surface, the new one is gated fresh.

4 THE KERNEL, MADE SOLID FOUR WAYS

The kernel is the fixed part: the gate, the sealed runner, the result order, the registry loader, and the driver that plays, regrades, and renders — five stdlib-only Python modules of 2,654 lines at the tag (`driver.py` 689, `gate.py` 1,238, `registry.py` 191, `results.py` 419, `runner.py` 85). It is the one piece of code the gate cannot judge, because it is the gate; it is generated like everything else; and so four disciplines stand in for the gate (Table 1), each worded no stronger than what it verifies. The kernel computes no truth of its own: it runs judges, compares bytes, orders results, and draws the log. A kernel bug can lose a result or misgrade one; that it can never forge one — no record reaches *certified* unless an interpreter run where the question lives passed — is stated in the specification and falsified by test (`test_forge.py`): on a toy registry bootstrapped from empty through the real gate, a search that prints a witness together with the words `"grade": "certified"` never becomes a result, because its witness does not replay; a bogus certificate floors at *claimed*; a broken carry-back only loses the witness; and `regrade` lifts a stored proof to *certified* by check time alone. Section 7.1 asks the same question of the real registry.

The half that is mathematics is proved. A *key* is $(\mathit{level}, \mathit{bound}, \mathit{grade}, \mathit{gap})$ with $\mathit{gap} \in \mathbb{N} \cup \{\bot\}$; write $k \prec k'$ for the lexicographic order in which a smaller finite gap is better and $\bot$ (no judgment ever ran) is worst, $k \preceq k'$ for $k \prec k'$ or $k = k'$, and $\mathit{best}(\ell)$ for the fold of a log ℓ that keeps the incumbent only while it is strictly better; the best of an empty log is below every key, and $\ell\,\ell'$ is ℓ followed by ℓ'.

Table 1. The four disciplines that stand in for the gate on the kernel, what each verifies, and what it does not. Sizes are lines at the tag.

discipline	verifies	does not verify
proof — Lean 4, 502 lines, core library only, toolchain pinned	the order on keys and the ratchet (Theorem 4.1); the residual (Theorem 4.2); axiom audit printed at every build: order lemmas axiom-free, fold and residual lemmas on propext and Quot.sound through core list lemmas	the Python that computes keys and residuals; `test_order.py` runs the same properties against `results.key` on random records
tests — `python3 -m kernel.tests`, 2,259 lines, nine modules, a toy registry	the gate on the registry's own controls (every stamp re-derived, every shipped mutant refused again) and on the toy (every way of failing it fails); the seal (blank `PATH`, scratch directory, no tool reachable, a hit wall a result); pins recomputed from their definition; the frame contract; board and graph regenerated byte-identically; the fault harness	inputs no pinned run exercises beyond the toy; a stamp's agreement is a floor, since a pair may join after its language was stamped
second lineage — `kernel/second/`, 727 lines, clean-room from the specification and the committed data	byte-agreement with the first kernel on `base`, `report`, and `graph` over every pinned run and a synthetic run holding the corners (`test_second`); an audit hook on `open` confirms it reads no file of the first kernel	`play`, `admit`, `regrade`, which it never runs; eleven listed ambiguities of the specification on inputs no pinned run exercises
content pins — sha256 over every entry, re-verified by the loader	that admitted bytes are the judged bytes; two entries claiming one name at one revision refused	anything about what the bytes mean

Theorem 4.1 (the order and the ratchet). *$\prec$ is irreflexive, transitive, and asymmetric. For all logs ℓ, ℓ': $best(\ell) \preceq best(\ell\,\ell')$; if $best(\ell)$ is settled then so is $best(\ell\,\ell')$; and if the two have the same level and bound, then the grade of the second is no lower and, at equal grade, its gap no larger. A record that differs from another only by a higher grade, or only by a smaller gap, is strictly above it.*

Theorem 4.2 (the residual). *Let $h_1, \ldots, h_n$ be the lineages of a route's hops, j the lineage of the judge that ran, and $residual(h, g, j) = \bigcup_{i \le g} h_i \cup j$. Then $residual(h, 0, j) = j$; $g \le g'$ implies $residual(h, g, j) \subseteq residual(h, g', j)$; for $g \le n$, appending hops upstream of the judgment leaves the residual unchanged; and every element of a residual belongs to the judge or to a hop, never to the search alone.*

The first clause of Theorem 4.2 is what makes *certified* route-independent; the last clause of Theorem 4.1 is why a regrade, which keeps the value, is a strict improvement exactly when it moves the grade or the gap.

The half that is operation is falsified, and doubled. The test suite is the gate turned on the kernel; one of its tests is how the blank `PATH` got into the specification — an unset `PATH` still found `/usr/bin/python3`. The second lineage is the accelerator discipline of Section 3.6 turned on the kernel: two implementations of disjoint descent that agree on every byte corroborate each other, as two judges do. Its README records the protocol — what was read, what was run (the first kernel only as a black box), one disclosure (an early traceback that named two source files; the first kernel now refuses a corrupt registry without one) — and where they disagreed while being built (the

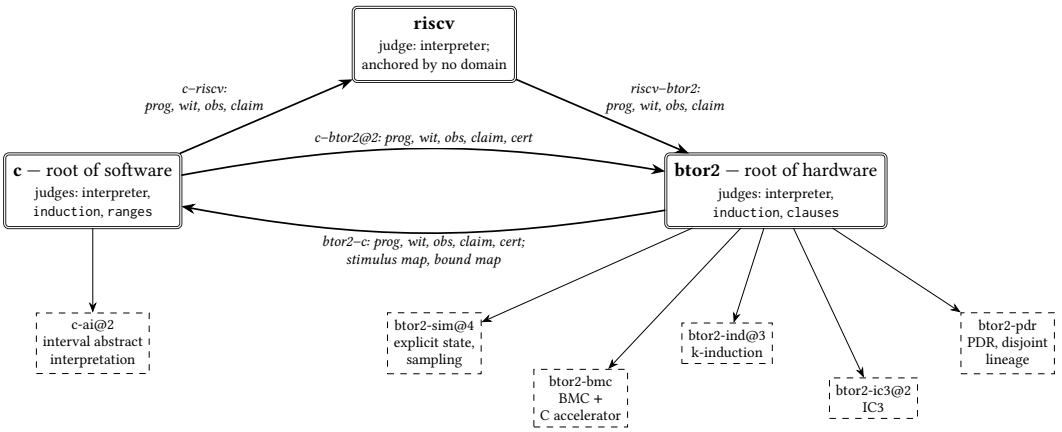


Fig. 4. The registry at the tag: three languages (double borders), four pairs (arrows, labelled with the artifacts they carry), six searches (dashed, at the language they live at), names at their highest admitted revision. Routes never revisit a language, so the two pairs between c and btor2 never play the round trip.

Table 2. The trusted base at the tag, as base prints it — judges only. Vectors are programs with pinned observables, or for a checker certificates that must discharge; controls are mutants the gate refused, or certificates that must not discharge. Sizes are lines of Python.

judge	kind	vectors	controls	lines	corroborated by
btor2@6	interpreter	18	10	390	hardware (3 anchors)
btor2:induction	checker	18	12	1,620	
btor2:clauses	checker	10	10	1,618	
c@7	interpreter	17	10	1,422	software (3 anchors)
c:induction	checker	46	25	4,534	
c:ranges	checker	42	21	3,935	
riscv@2	interpreter	20	12	603	no domain; its pairs' squares only

key's tie rule, the pin's exclusions, the binding rule for revisions) the specification was made to say which is right. Manual review is the last discipline, and the four above are what make it tractable: what must be read is the trusted base, printed by base, and the kernel's five modules, and the reviewer reads theorem statements, tests, and a second implementation's disagreements rather than a story.

5 WHAT WAS ADMITTED

Everything under registry/ at the tag entered through the gate of Section 3.7, in the order the conjecture order prescribes (Table 4), and every entry carries its stamp. Figure 4 draws the registry; Table 2 is the trusted base as the kernel prints it.

Languages and pairs. BTOR2's interpreter is the hardware floor; C's is the preprocessor-free ILP32 integer fragment SV-COMP's loop and bit-vector tasks are written in, total and deterministic; RISC-V's is the middle vertex, anchored by no domain. The three pairs of the triangle, c–btor2, c–riscv, riscv–btor2, each carry programs, witnesses, observables, and claims, with mutants per artifact kind and exchange rates of 3.65, 5.37, and 3.13 target bytes per source byte; c–btor2@2 also carries certificates, and C's induction checker builds its machine by the RISC-V road, so a proof

that travelled the pair is certified by an encoding the pair never touched. The reverse edge `btor2-c` is the one pair on the frame contract of Section 3.2: every state a global of the smallest unsigned container holding its width, one statement per node, `ite` branch-free, a `bad` at machine frame t observed at C frame $n_0 + n \cdot t + bad$; seventeen corpus machines, six certificates carried both ways, thirteen mutants refused, 4.19 bytes per byte; array sorts, widths above 64, and non-constant initial values refused loudly.

Certificate forms and searches. BTOR2 judges `induction` (bit-invariants and k-induction) and `clauses` (a conjunction of clauses over state bits, as property-directed searches converge to); C judges `induction` and `ranges` (per program point a box over the scalar slots, unlisted points asserted unreachable). At BTOR2 live `btor2-sim@4` (explicit-state enumeration, guided sampling when the input space is large, arrays as sparse maps), `btor2-bmc` (bit-blasting per frame to an and-inverter graph decided by a generated CDCL solver, an accelerator in C admitted by byte-agreement), `btor2-ind@3`, `btor2-ic3@2`, and `btor2-pdr` [14], written from nothing it shares with `btor2-ic3` — its own parser, bit-blaster, and SAT solver — so that its lineage is disjoint from every earlier search's; at C lives `c-ai@2`, interval abstract interpretation that writes a `ranges` certificate or an honest partial and never a bare claim.

Revisions. Fifteen language entries, five pair entries, thirteen search entries, and two domains stand in the tree; a name binds to its highest revision, and the earlier revisions are the system's history without an edit: `btor2-ind@2` and `btor2-ic3@2` blasted the signed division family after thirteen SV-COMP programs and four HWMCC machines crashed the first revisions; `btor2@6`, `c@6`, `c@7`, and `riscv@2` add vectors and controls alone after the measurement of Section 7.1, and change no line of any judge.

The generator. Every implementation in the registry, and the kernel, was written by language-model agents at the command line the loop describes (Section 6); the entries declare four generator tags among their lineages. A human chose the benchmarks, set the goal of each campaign, and pulled the plug. Two things the tree does not record limit what can be said about the generator: an entry that fails the gate leaves no trace, so the rate at which candidates were refused during development is not known; and nothing was generated twice from one instruction, so the generator's variance is not measured. Section 7.1 measures what can be measured after the fact.

6 PUSHING THE FRONTIER

The frontier of a benchmark is pushed by one loop (Fig. 5), run by a player who never writes a result: a language model, or a human at the same command line. This section is the loop as an operating procedure, and the record of the turns that produced the tree at the tag.

What an entry ships. An entry is a directory the player writes and `admit` adjudicates. A search ships `manifest.json` (kind, language, name, revision, predecessor hash, lineage, the observables it targets), `solve.py`, an optional trust-inert `ledger.py`, a `corpus/` of programs with asks whose evidence must judge valid, and `controls/` of mutants that must fail — `btor2-sim@4` ships seven corpus questions and three mutants: one drops array cells, one lies, one overclaims. A pair ships `T.py`, the carry-backs it declares (`lam_wit.py`, `lam_cert.py`, on the frame contract `lam_in.py` and `lam_bound.py`), its direction, kept observables, and corpus, with mutants per carried artifact. A language ships its interpreter, vectors with pinned observables, mutants, and under `evidence/<form>/` a checker with example certificates and mutant certificates. The gate stamps what it checked into the manifest's `admission` field, and the stamp is the only self-description an entry is allowed.

What the player reads. `report` renders the board, one row per question, from the log:

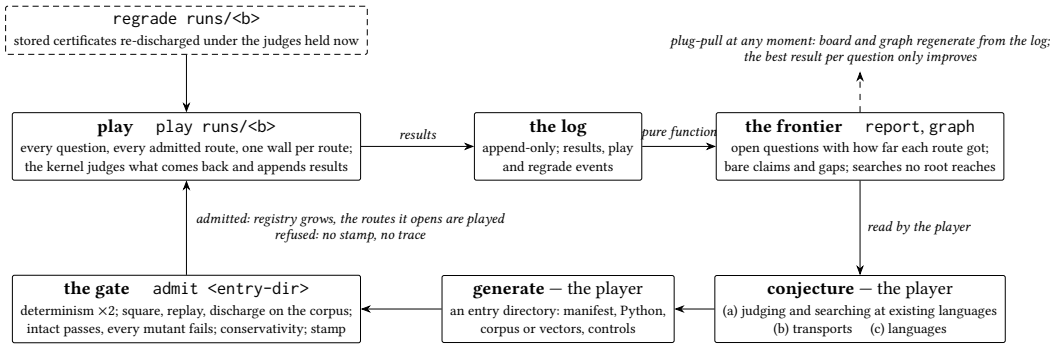


Fig. 5. One turn of the loop, and the kernel command behind each step. The player — a language model or a human at the same command line — reads and generates; only the kernel writes results, only the gate writes the registry, and neither command takes a conjecture as input. Table 4 lists the turns that produced the tree at the tag.

Table 3. The kernel's commands. Each reads the registry and a run and writes one thing; none takes a conjecture as input.

python3 -m kernel.	reads	writes
driver admit <entry>	the entry directory, the registry it would join	a stamp into the entry (admission evidence, content pin), or nothing
driver play <run> [-wall S]	the pinned benchmark, every admitted entry; routes enumerated breadth-first over the whole registry, never revisiting a language	one result per (question, route) appended to the log, under one play event; after an admission only the routes it opens
driver regrade <run>	stored certificates in the log, the judges held now	re-derived results appended; check time only
driver report <run>, graph <run>	the log	frontier.md, frontier.dot: pure functions, byte-identical on every call
driver base	the registry	the trusted base, a list
tests; second.driver; faults	the toy registry; the committed data; the registry	the kernel's falsification; the second lineage's byte-agreement; the fault-injection deposit

question	best result	grade	gap	trust	route	s
open1	all (bound 6)	claimed	—	btor2-bmc-g1 fable5-e67f . . .	btor2-bmc	0.2
open26	partial (nested arrays: outside this fragment)		—	—	btor2-sim	0.0
unsafe1	witness (depth 6)	certified	0	btor2-cert-g1 btor2-interp-g1	btor2-bmc	0.5

Three rows of hwmcc24-arrays say what the frontier is made of. The first is a bare claim to bound 6 on an unbounded ask, settled by nothing, its trust column the whole chain: it asks for a certificate printer at BTOR2, or a checker for a form some search already writes. The second is a partial whose progress note names the refusal, which is the (a)-move written out. The third is settled and needs nothing; its trust column holds only BTOR2's judges. The frontier's second reading is structural: which roots reach which searches, and the one pair each unreached search is missing.

Table 4. The turns of the loop that produced the tree at the tag, in order: what the frontier showed, the move in the conjecture order, the entry the gate admitted, and what the next play did to the boards (Fig. 6). Routes are played with a 60 s wall unless stated.

the frontier showed	move	admitted	the boards after the play
an empty registry and HWMCC'24 machines	—	domain `hardware` (root, labels, official verdicts); interpreter `btor2`; search `btor2-sim`	`hwmcc24-mini` 6 of 74: 3 witnesses by replay, 3 claims
68 open; enumeration exhausts nothing large	(a)	`btor2-bmc`, with a C accelerator byte-agreed on its corpus	42 of 74: 9 witnesses, 33 bare claims
33 bare claims, nothing judges them	(a)	forms `induction`, `clauses` with checkers (revision of `btor2`); searches `btor2-ind`, `btor2-ic3`	43 of 74: 14 certified ∀ at gap 0, 20 claims
SV-COMP tasks, no root reaches the hub	(b)	domain `software`; interpreters `c`, `riscv`; pairs `c-btor2`, `c-riscv`, `riscv-btor2` (30 s wall)	`svcomp25-mini` 26 of 79: 14 witnesses, 3 *checked* at gap 1, 9 claims
3 proofs *checked* at gap 1, residual naming `c-btor2`	(b)	checker `c:induction` (revision of `c`); certificate carry-back on `c-btor2@2`; then `regrade`	the 3 lifted to gap 0 in under a second each, nothing re-solved
signed division crashes 13 programs and 4 machines	(a)	`btor2-ind@2`, `btor2-ic3@2`, byte-identical on the predecessors' corpora	re-played through the revisions
software claims with no printer at C; hardware questions reach one lineage	(a), (b)	form `ranges` with checker; search `c-ai`; reverse edge `btor2-c` on the frame contract	`svcomp25-mini` +1 certified; 74 partials over the reverse edge (41 idiom-blind, 17 width-refused, 12 existential, 4 budget)
ten open universal questions on `hwmcc24-mini`	budget	`btor2-ic3` alone, 600 s wall	one proved (gap 0 in 30 s), six killed at twice the wall: search-limited
two larger rungs pinned; arrays refused by sim, IC3, the reverse edge	—	first plays of `hwmcc24-mid`, `hwmcc24-arrays`	47 of 80 (12 certified); 1 of 55
43 array partials naming the refusal	(a)	`btor2-sim@4`: arrays as sparse maps	`hwmcc24-arrays` 12 of 55, eleven witnesses by replay
20 and 27 bare claims on the hardware rungs; one lineage of judges and searches	(a)	`btor2-pdr`, disjoint lineage: own parser, bit-blaster, SAT solver	45 of 74 (24 certified), `hwmcc24-mid` 19 certified, software +1; corroborated fires on 29 and 22 verdicts
the reverse edge's branch-free `ite`, opaque to a box	(a)	`c-ai@2`: reads `ite` as a selection, adds a case split	nothing: 154 partials
six laxer judge mutants (Section 7.1)	revision	`btor2@6`, `c@6`, `c@7`, `riscv@2`: vectors and controls only, expectations from outside oracles	boards unchanged; judge mutants refused 348 → 453 of 700
39 open existential questions the competitions solved	budget	the competitions' results and a host calibration pinned as testimony; `btor2-bmc` alone, 900 s wall	48 of 74 (two witnesses, one covering claim), 28 of 79 (two witnesses); memory stops the middle rung, a 300-frame ceiling software (Table 5)

The conjecture order. Conjecture in this order — semantics first, then syntax: (a) *new judging and searching at existing languages* — a search, a certificate form with its checker, an accelerator for a proven bottleneck; (b) *new transports* — pairs, and carry-backs retrofitted onto admitted pairs by revision; two (b)-moves are canonical, a certificate carry-back added to a proven translator, and a translator from a root that reaches no hub to the nearest one, which is the entry move of every new domain and inherits every search at the hub for the price of one translator; (c) *new languages*, only when a translation or solving move keeps winning ad hoc. Then generate, admit, re-play the routes the entry opens. The kernel ships with zero languages, transports, and domains; presented a benchmark, the first admissions are the domain — root name plus anchors — then the root's interpreter, then a naive search in pure Python. Witnesses need no declared form, so existential questions settle *certified* with a single admitted interpreter; the first checker arrives when universal questions need better than *claimed*. Table 4 is the record that this is how the tree was populated, and Fig. 6 is what each turn did to the boards.

Why the searches are not competitive, yet. The generated searches implement bounded model checking, k-induction, IC3, PDR, and interval abstract interpretation in Python, each with its own generated SAT solver; the solvers whose verdicts sit beside the boards embed decades of tuning,

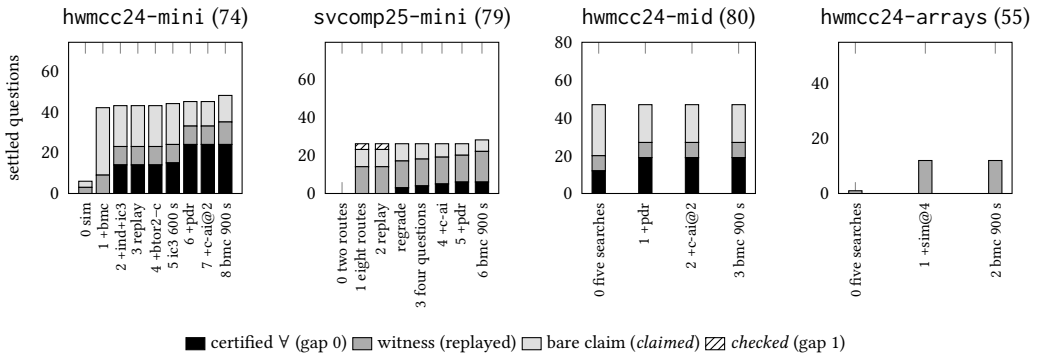


Fig. 6. The board after every play and regrade event of the four pinned runs, read from the logs; the axis top is the number asked, so the blank above a bar is the frontier. Each bar is labelled with the iteration and the entry that opened it. Settled counts move rarely; grades move more: `btor2-pdr` lifts bare claims to gap 0 on both hardware rungs, and `regrade` lifts three *checked* proofs on software without re-solving; the last bar of each run is the 900 s pass of Table 5.

and the generation rule (Section 3.6) forbids wrapping them. Three facts about the design fixed the order in which things were asked of this commit: the frontier is a function of level, bound, grade, and gap, never of cost, so a faster search moves no question unless it also settles or regrades one; every performance move the design admits is judged — an accelerator by byte-agreement, a hint by the judgment its output faces, a regrade by the checker — so performance can be pursued after the trust claims are measured, by the same loop, with no revision to anything the fault injection examined; and that measurement had to come first, because its truth set is the intact judges and the boards' certified answers, and a board of fast bare claims would have measured nothing.

The oracle column. What the design reserves for a comparison is testimony: the per-question results a competition archive publishes are pinned beside its verdicts, one calibration run of a pinned solver on the playing host makes the ratio honest, and the board draws each question's cost against the best oracle's — recorded, never ranked, never a grade. Table 5 draws it at the tag, with the one budget move made against it: the 39 open existential questions the competitions solved, played through `btor2-bmc` alone with a 900 s wall under a guard that kills a search above 6 GB on a 17 GB host (the competitions allowed 120 and 15). Where a question is settled, the generated search cost a median of 2 to 42 times the best official solver's time. The pass settled five of the 39 — four witnesses certified by replay and one covering claim — and on those the pinned solver on the same host, with the same bound, is 6 to 14 times faster. What ended the other routes is named per route, and it is not the wall: a conflict budget that derives from the declared wall at the reference's pace and that the C accelerator spends in 300 s; memory, on the middle rung and on software; a ceiling of 300 frames for unbounded asks, one CFG node per frame; and the structural ceilings of the eager array reduction. The host calibration reads the ratio: with the question's bound, `btormc` answers 35 of the 46 existential hardware questions within 900 s on this host and 6 of the 16 the pass played, and `cbmc` with doubling unwindings answers 20 of the 37 software ones and 6 of the 23 the pass played. Two bounded claims sit beside unbounded verdicts, one on each hardware rung, and `btormc` confirms within 5 s for both that the bad state lies beyond the pinned bound 30. The column names the next (a)-moves as the frontier names the rest: a budget calibrated at admission to the accelerator's measured pace, a deeper ceiling with one basic block per frame, a lazy array reduction, eleven cones above `btor2-pdr`'s cap, seventeen translations refused above 64 bits.

Table 5. The oracle column at the tag. Settled: questions with a best official time, and the median of the best result's cost over it; open, solved: open questions some official solver answered, and the median of its time. The 900 s pass: questions and routes played, what settled, what ended the other routes; host: the pinned solver on the playing host with the question's bound and the same wall, answered/asked, on all existential questions and on the pass's.

	settled	open, solved	the 900 s pass			
board	*n* ratio	*n* official	q/routes	settled	the other routes	host
`hwmcc24-mini`	48 42	19 0.7 s	6/6	3: 2 witnesses, 1 covering claim	3 bare claims (bounds 1, 15, 17), budget spent by 300 s	`btormc` 11/15, 2/6
`svcomp25-mini`	28 3.0	51 0.2 s	23/46	2: witnesses, depths 50, 63	13 at the 300-frame ceiling; 20 out of memory (19 by the guard); 1 past twice the wall; 10 bare claims	`cbmc` 20/37
`hwmcc24-mid`	47 10	25 30 s	8/8	0	6 out of memory (4 by the guard); 1 past twice the wall; 1 bare claim (bound 15)	`btormc` 11/17, 3/8
`hwmcc24-arrays`	12 2.1	5 0.5 s	2/2	0	2 bare claims (bounds 18, 7), structural ceilings within 2 s	`btormc` 13/14, 1/2

7 MEASUREMENTS

Every number in this section is read from the tree at one tag. Three things are measured: what faults placed deliberately do to the gate and to the grades (Section 7.1); what it took to close what they found (Section 7.2); and what the system settles on four pinned benchmarks (Section 7.3).

7.1 Fault injection

Every entry ships its own mutants and the gate refuses them; that says the controls can fail, not how much they see. We therefore mutate the registry ourselves, in the manner of mutation testing [13, 18]. A *mutant* is one executable with one classical fault: an operator replaced, `and` and `or` exchanged, a negation dropped, a branch condition negated, a constant moved by one or flipped, or an effect-free statement removed. Sites are enumerated over the syntax tree of every executable a name binds to — the seven judges and the eighteen transports (twelve translators, carry-backs, and maps of the four pairs; six searches) — and one hundred per file are drawn by a hash of the site, so the sample is reproducible; two small files have fewer sites, giving 700 judge mutants and 1,672 transport mutants. A mutant is staged in a scratch copy of its entry and run sealed, under a wall of four times the intact entry's longest run and a memory cap; the registry and the logs are never written, and each experiment first runs the intact file and its reparsed identity, which must behave as the entry does.

At the gate. A mutated entry faces what an entry of its kind faces at admission, in order, and the first refusal is recorded: its *own* vectors, corpus, and controls; *conservativity*, byte-agreement with its predecessor on the predecessor's checkable surface; and, for a judge only, its *dependents* — the admission of every pair and search that runs the judge, re-run against the mutant, which is what Section 3.1 means by a judge being corroborated after it entered. Of the 1,672 transport mutants the gate refuses 736 (694 by the entry's own gate, 42 by conservativity) and passes 936, of which 528 sit on lines no admission input executes. That is the design's free surface: what a transport writes still faces judges at play time, and the next experiment measures what the 936 can do. Of the 700 judge mutants the gate refuses 348 — 292 by the judge's own vectors and controls, 26 by its predecessor, 30 by dependents (Table 6, left). Two things in that count bear on the design. A revision is a differential oracle: 25 faults in BTOR2's interpreter pass its vectors and are caught only by disagreement with the previous revision. And the claim that RISC-V, anchored by no domain, is held in place by the pairs on either side of it has a number: eleven faults in its interpreter pass

Table 6. The judges under fault injection, 100 sampled mutants each. *Before*: at the revisions the boards of Section 7.3 were first played under. *After*: the identical mutants against the vector-only revisions of Section 7.2. Statements: the share of the judge's statements executed by any admission input. Laxer: gate survivors that discharge a certificate the intact checker refuses, on the pool of the probe.

	before			after		
judge	statements	refused	laxer	statements	refused	laxer
`btor2` interpreter	90%	80	—	96%	98	—
`btor2:induction`	54%	33	4	61%	43	0
`btor2:clauses`	44%	26	1	52%	42	0
`c` interpreter	79%	61	—	89%	83	—
`c:induction`	63%	37	1	69%	43	0
`c:ranges`	70%	49	0	75%	55	0
`riscv` interpreter	76%	62	—	93%	89	—
all seven, of 700		348	6		453	0

Table 7. 16,720 plays of 1,672 transport mutants forced past the gate, each record held against the known answer. *Wrong*: the record settles the question the other way — a universal claim covering the ask where a witness is known, or a witness within the bound a question is known safe to.

				wrong, by the grade it carries		
mutants of	same	lost	gained	*claimed*	*checked*	*certified*
the pairs' transports (1,072)	9,387	1,312	14	1	6	0
the searches (600)	4,769	1,056	49	126	0	0
all (1,672)	14,156	2,368	63	127	6	**0**

its own vectors and are refused only by those pairs' squares. The 352 survivors are the trusted base's exposure, and they sit where the judges are large: the interpreters refuse 61 to 80 of 100, the checkers 26 to 49, with admission executing 44% to 70% of a checker's statements.

With the gate taken away. The design's central claim is not about the gate. It is that a transport, being judged, can lose a grade and never forge one, and that the residual names what a result rests on (Section 3.4). To test it we remove the gate: each of the 1,672 transport mutants is put where the intact file stood, as if admitted, and the kernel plays the routes through it on questions whose answer is known — the labelled corpus questions of the searches, and questions of the pinned runs for which the board holds a replayed witness or a universal answer certified at gap 0 (the truth presumes the intact judges; the competitions' verdicts agree on every such question). For each transport ten (question, route) pairs the intact entry plays within two seconds are chosen by hash from four kinds, so that a wrong answer has somewhere to show: unsafe and answered by a witness, safe and answered by a judged proof, unsafe and unanswered, safe and unanswered — 180 pairs, half of them unsafe. Each record the kernel writes for a mutant is then held against the truth and against the intact route's record (Table 7).

No play leaves a wrong record *certified*. The 127 wrong *claimed* records are what the lowest grade is for: a search whose comparison is inverted answers "safe forever" on an unsafe machine, nothing judges the claim, and its residual is the whole chain, the search included. The six wrong *checked* records are the instructive ones. All come from mutants of the C-to-BTOR2 translator on

one unsafe SV-COMP task: the mistranslated machine really is safe, the property-directed search proves it, BTOR2's checker discharges the certificate — and the certificate, carried home, fails to discharge at C, so the gap stays at one and the residual names the translator. In all 133 wrong records the residual contains the faulty entry's whole lineage. Read against the first experiment: of the 736 mutants the gate refuses, 46 would have answered wrongly on some question and 506 would have lost an answer; of the 936 it passes, 832 change nothing on any played question, 90 lose an answer, 7 only gain, and 7 answer wrongly — one of them a translator mutant behind a wrong *checked* proof. The gate missed those seven and the grade still said what each answer rested on.

Which way a surviving checker leans. For a judge there is no judge downstream, so a surviving mutant has to be classified another way. A checker mutant the gate passes is the same judge, a stricter one — it refuses something the intact checker discharges, which costs answers and no trust — or a *laxer* one, which discharges something the intact checker refuses; only the last is a hole. We hold each of the 255 surviving checker mutants against the intact checker on a pool of up to sixty (program, certificate) pairs per checker, drawn from every certificate of its form in the tree — vectors, controls, what the searches write on their corpora, what the pairs carry, the logs of the pinned runs — each also set against programs it was not written for and edited once. On that pool 245 agree with the intact checker everywhere, 4 are stricter, and 6 are laxer: in BTOR2's checkers, `neq` bit-blasted as `eq`, a signed comparison reading one operand twice, and three faults in the embedded SAT solver's propagation and assumption handling; in C's `induction` checker, a zero-extending shift dropped from the machine it builds. "Same" is a statement about the pool, not a proof of equivalence: most of the 245 sit in code that no certificate we could find reaches.

7.2 Closing the survivors

A survivor on a line that admission executes names a missing test, and the design has one way to add a test: a revision (Section 3.7). The four revisions made after the measurement change no line of any judge. What they add is vectors and controls, and since a vector's expected value written by reading the judge would test nothing, every expected value was computed from the relevant standard by a small generator and then put to an *oracle* in the sense of Section 3.6, a tool that runs outside the seal and never enters the registry:

- BTOR2: 301 checks, each one operator on constants against the value SMT-LIB's definitions give, the conjunction of a family as the bad property. BtorMC [32] finds every check reachable as expected and unreachable negated; the same tables, states frozen at the constants, are certificate vectors for both checkers, the invariant pinning every state bit and its control that invariant made wrong.
- RISC-V: 896 checks, each one instruction or short idiom against the value the specification gives, assembled bare-metal and run on the executable of the Sail model [3]. Every table succeeds as written, and with any one expected value moved by one fails with that check's number.
- C: assertion programs for the interpreter, and for the checkers 37 programs that never fail — operator tables at the corners of each integer type, division in programs of two variables, identities over all pairs of bytes — with 651 checks that clang confirms as written and refutes with any one expected value moved. Each `ranges` certificate is what `c-ai` wrote, each `induction` certificate k-induction at the smallest k that discharges.

No oracle disputed a judge: on every new check the generated interpreters and checkers were right, and the corners had simply never been observed. (The oracles did catch five wrong expectations in our own generator, which is what they are for.) Where the fragment stipulates a meaning the standard leaves undefined — a shift count at or beyond the width, a zero divisor — the vector

Table 8. The pinned benchmarks and the boards at the tag. Settled: a replayed witness (certified at the root), or a universal claim covering the ask, certified at gap 0 or bare (*claimed*). Corroborated: settled verdicts on which two results of disjoint lineage agree. Official verdicts are the competition archives'; unsolved means no official solver answered.

benchmark	asked	settled	of which			corr.	open	open, officially	
			cert. ∀	witness	claim			unsolved	solved
`hwmcc24-mini`	74	48	24	11	13	29	26	7	19 (16 unsat, 3 sat)
`svcomp25-mini`	79	28	6	16	6	0	51	0	51 (30 true, 21 false)
`hwmcc24-mid`	80	47	19	8	20	22	33	8	25 (17 unsat, 8 sat)
`hwmcc24-arrays`	55	12	0	12	0	0	43	38	5 (3 unsat, 2 sat)

pins the stipulation and no oracle is asked. Table 6, right, repeats the first experiment on the identical mutants. The interpreters now refuse 98, 83, and 89 of 100, and every one of the thirty that remain is, by inspection, an equivalent mutant, a removed refusal — which a vector, a program the interpreter must run, cannot observe — or one ordering of nested array values no vector builds. The checkers refuse 42 to 55 of 100; all six laxer mutants are refused by the new vectors, and none of the 217 mutants that still survive is laxer on the pool. What is left sits mostly in code no certificate check reaches — each checker embeds whole an interpreter, an array engine, or translator cases its own road never emits — weight a judge should not carry, and the next thing the measurement asks for. The vectors are not free: admitting C's language entry now takes a little over two minutes where it took under one.

7.3 The boards

Protocol. Four benchmarks are pinned, every program by sha256 (Table 8). Three are tiers of the HWMCC'24 bit-vector track [17, 32]: `hwmcc24-mini` (74 machines, 23% of the track), `hwmcc24-arrays` (55 machines with array sorts), and `hwmcc24-mid` (80). The fourth, `svcomp25-mini`, holds 79 ReachSafety loop and bit-vector tasks from SV-COMP 2025 [9], 31 of them tasks that at most three of the verifiers in the archive answered correctly. An ask is `exists bad` at a bound, or `forall bad` at a bound or at ∞. A play runs every question along every admitted route whose fragment admits it, one wall per route — 60 s, except two early software plays at 30 s and one pass at 600 s — and a search that runs to twice its wall is killed and booked as a partial saying so; after an admission only the routes the new entry opens are played.

What the plays found. Figure 6 shows the board after every play and Table 4 the entry behind each; three things in that record bear on the design. Twelve of the fourteen first software witnesses walked home across both bridges of the triangle, found at BTOR2 on the route C to RISC-V to BTOR2 and replayed by I_C, and three universal answers on software first *checked* at gap 1 were lifted to gap 0 by `regrade` alone once C shipped its own checker and the pair a certificate carry-back. The negatives are in the log with their reasons (Table 4): 74 partials over the reverse edge, the array rung's first play settling 1 of 55 with three searches refusing array sorts, IC3 with a ten-minute wall proving one of ten and killed at twice the wall on six. One turn of the loop answered them with three (a)-moves: `btor2-sim@4` moved the array rung to 12 of 55 by eleven replayed witnesses; `btor2-pdr` certified 22 universal answers on `hwmcc24-mini` and 17 on `hwmcc24-mid`, one newly settled and fifteen lifted from bare claims, plus one on software, while its 79 routes over the RISC-V road are all partials and eleven machines of the middle rung exceed its cone cap; and because its lineage is disjoint from every earlier search's, the *corroborated* flag fired for the first time, on 29

and 22 verdicts — two searches, not two judges; no software verdict is corroborated, every route to a C question sharing the C front end; `c-ai@2` moved nothing.

Against the official verdicts. The competitions' verdicts are anchors, recorded per question with provenance, never ranked, never a judge. All 48, 28, 47, and 12 settled questions agree with them, with two rows that are not the same answer: an existential question on each hardware rung settled in the negative by a bare claim to bound 30 that official solvers record as satisfiable at a depth the archive does not publish — a bounded claim beside an unbounded verdict, not a contradiction (Table 5). Of the open questions, 53 on the hardware boards were solved by no official solver and 100 across the four boards were, among them every open software task. No settled question disagrees with the archive, and none settled here is recorded there as unsolved.

7.4 What the numbers say

Four readings are supported, and no more. The no-forge claim survives a test designed to break it: with the gate removed, 1,672 faulty transports produce wrong answers only at the two grades that say "not judged at home", each residual naming its cause. The trusted base is measurably the weak part and measurably improvable: half of the sampled judge faults passed admission, six were real holes, and vector-only revisions with outside ground truth closed all six and raised the refusal count from 348 to 453 of 700. The geometry does what it was built to do on real questions: 49 universal answers are *certified* at gap 0 with the home judge alone in their residual, sixteen lifted from bare claims by a later search, three by a regrade that re-solved nothing; the 39 covering claims are *claimed* because nothing judged them. And the system is far from the solvers it records beside its boards — it settles no question they left unsolved and leaves 100 open that they closed (Table 5 and Section 6).

8 RELATED WORK

Generated verification infrastructure. Language models are used to write proofs and repair them until a checker accepts them [15, 43, 44], to propose invariants for a verifier to confirm [33, 42], to synthesize code together with its specification and proof [1, 28], and to keep code, documentation, and tests consistent under a checker's eye [37]. In every case the checker is fixed and trusted and the model fills one slot; here the model fills every slot and trust moves to a property no slot can assert, where along a route an interpreter run judged the evidence. Kirsch [20] gives a calculus of directional squares, a componentwise meet for composed translations, and the asymmetry between replayed witnesses and universal claims, for a platform of hand-wrapped engines; the design here consumes that calculus and nothing else of that platform.

Translation validation and verified compilation. Checking a translation per program rather than proving the translator [31, 34] is the square of Section 3.2, with two differences: both sides are generated interpreters rather than a trusted semantics, and the check is on a shipped corpus at admission with mutants that must fail, not on every compilation. Verified compilers [23, 24] and verified translation validators [35, 40] buy a stronger statement at the cost of a proof per translator; Alive2 [25] bounds the check as a bound cap bounds a claim here. The system proves no translator; what it adds is that a result's grade never depends on whether a translator was right, only on whether a judge ran.

Certificates, witnesses, and proof formats. Certifying model checkers [29], certified hardware model checking [46], verification witnesses [7, 8], proof-carrying code [30], certifying algorithms [26], and checked proof formats [2, 38, 41] share the move that an untrusted producer emits an object a small checker validates. Here the checker is generated too, admitted by its own controls,

and the object's grade records how many translations lie between the checker that validated it and the question. Certified abstract interpretation [5, 19] produces or verifies the kind of certificate `c:ranges` checks; the difference is that the analysis here is untrusted and the checker is what was gated.

The searches. The generated searches implement known algorithms: bounded model checking [10], k-induction [36], IC3 and property-directed reachability [11, 14], interval abstract interpretation [12]; Table 5 records their distance from the competitions' solvers. Btor2C [6] translates BTOR2 machines to C so that software verifiers reach hardware questions; the reverse edge is the same move gated in both directions, a negative at the tag.

Mutation analysis. Mutation testing measures a test suite by the faults it detects [13, 18]. Section 7.1 turns it on a verification system in three ways: the "test suite" a mutant faces is the admission gate; a mutant of an untrusted component can be executed past the gate and its outputs scored by the grade the system itself assigns; and a surviving mutant of a checker is classified by direction, since only one direction is unsound.

The trusted base. That trust cannot be bootstrapped from a component one did not judge is old [39]; the LCF discipline of a small kernel every proof must pass [16] is the shape of the trusted base here, the kernel's own standing established by proof, tests, a second lineage, and pins, not by being small. Differential testing [27, 45] and N-version programming [4] supply the second-lineage discipline; independently written versions fail together [22], so the corroborated flag requires declared disjoint descent. Rotor [21] fixes one translation from RISC-V to BTOR2 by hand; the pairs through RISC-V are translations of that kind generated and gated, the ISA semantics generated too, with Sail's model [3] as an oracle for the interpreter's vectors.

9 LIMITATIONS AND CONCLUSION

The limits are the ones the tree states. The square is empirical: a pair is admitted when its squares close on its corpus, and a corpus program is not every program; the mutants make the corpus falsifiable, not complete. The judges are not small — C's two checkers are 4,534 and 3,935 lines — and after every revision (Section 7.1) 247 of 700 sampled judge mutants still pass admission, and for the 217 checker mutants among them the evidence that they are harmless is a pool of certificates, not a proof. The corroborated flag has fired between two searches of disjoint descent and never between two judges: every answer still rests on one lineage of interpreter and checker per language. Candidates the gate refused left no trace, so the generator's failure rate is not measured. The per-domain ledger is specified and not drawn; the oracle column reads that the generated searches are weak beside the competitions' solvers: no board holds a question they left unsolved.

Within those limits, a verification system can be built entirely from generated, untrusted text and judged whole: one gate admits everything against its own controls, one trust event — an interpreter run — is the only source of truth, and a result's grade is the distance from the question to the last such run. The claim that follows from the design — a faulty transport can cost an answer and cannot forge one — held on every one of 16,720 plays of faulty transports with the gate removed. The claim that does not follow from it — that the judges deserve their place — failed in six places, each found by the system's own instruments and closed by tests whose answers came from outside. What the frontier asks for next is smaller judges, a second lineage of them, and the bottlenecks Section 6 names. A domain of mathematics, physics, chemistry, or biology enters as hardware and software did — its root's interpreter and one pair to a hub, or a new hub with that domain's solvers as searches, trusted no more than the generated ones — and from then on every search reaches it and a question carried in is reasoned about across domains, at a cost not measured here.

DATA AVAILABILITY STATEMENT

The system is one repository, and the paper describes one frozen commit of it: the kernel and its tests, the second lineage of its pure half, the Lean development with its axiom audit, every admitted entry of the registry with the evidence of its admission, the oracle packs with the provenance of every expected value, among them the competitions' published result tables and the host calibration of their solvers, the four pinned benchmarks with their append-only logs, and the fault-injection deposit. Every number in the paper is read from that commit: the boards regenerate byte for byte from the logs, the fault-injection tables from the deposit, and the trusted-base table from the registry, each by one command of the kernel's driver, and a table in the repository names the file each number came from. The repository is public; its address is withheld here for anonymity and will be given in the final version. The artifact will be submitted for artifact evaluation; the outside tools that confirmed the oracle packs' expected values (Section 7.2) are not part of it.

REFERENCES

[1] Pranjal Aggarwal, Bryan Parno, and Sean Welleck. 2025. AlphaVerus: Bootstrapping Formally Verified Code Generation through Self-Improving Translation and Treefinement. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 587–615.

[2] Bruno Andreotti, Hanna Lachnitt, and Haniel Barbosa. 2023. Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. In *TACAS*.

[3] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 71:1–71:31.

[4] Algirdas Avižienis. 1985. The N-Version Approach to Fault-Tolerant Software. *IEEE Transactions on Software Engineering* SE-11, 12 (1985), 1491–1501.

[5] Frédéric Besson, Thomas Jensen, and David Pichardie. 2006. Proof-Carrying Code from Certified Abstract Interpretation and Fixpoint Compression. *Theoretical Computer Science* 364, 3 (2006), 273–291.

[6] Dirk Beyer, Po-Chun Chien, and Nian-Ze Lee. 2023. Bridging Hardware and Software Analysis with Btor2C: A Word-Level-Transition-System-to-C Translator. In *TACAS (LNCS)*.

[7] Dirk Beyer, Matthias Dangl, Daniel Dietsch, Matthias Heizmann, Thomas Lemberger, and Michael Tautschnig. 2022. Verification Witnesses. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022).

[8] Dirk Beyer, Matthias Dangl, Daniel Dietsch, Matthias Heizmann, and Andreas Stahlbauer. 2015. Witness Validation and Stepwise Testification across Software Verifiers. In *ESEC/FSE*.

[9] Dirk Beyer and Jan Strejček. 2025. Improvements in Software Verification and Witness Validation: SV-COMP 2025. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Part III (LNCS, Vol. 15698)*. 151–186.

[10] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. 193–207.

[11] Aaron R. Bradley. 2011. SAT-Based Model Checking without Unrolling. In *VMCAI (LNCS, Vol. 6538)*. 70–87.

[12] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *POPL*. 238–252.

[13] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (1978), 34–41.

[14] Niklas Eén, Alan Mishchenko, and Robert Brayton. 2011. Efficient Implementation of Property Directed Reachability. In *FMCAD*. 125–134.

[15] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *ESEC/FSE*.

[16] Michael J. C. Gordon, Robin Milner, and Christopher P. Wadsworth. 1979. *Edinburgh LCF: A Mechanised Logic of Computation*. LNCS, Vol. 78. Springer.

[17] HWMCC. 2024. Hardware Model Checking Competition 2024. https://hwmcc.github.io/2024/. Benchmarks and results archive.

[18] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (2011), 649–678.

[19] Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. 2015. A Formally-Verified C Static Analyzer. In *POPL*. 247–259.

[20] Christoph Kirsch. 2026. Untrusted Authors, Trusted Answers: A Calculus of Fidelity-Graded Translations. arXiv preprint arXiv:2607.14137, v2.

[21] Christoph M. Kirsch. 2017. Selfie and the Basics. In *Onward!*

[22] John C. Knight and Nancy G. Leveson. 1986. An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 96–109.

[23] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *POPL*.

[24] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115.

[25] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*.

[26] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161.

[27] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.

[28] Md Rakib Hossain Misu, Cristina V. Lopes, Iris Ma, and James Noble. 2024. Towards AI-Assisted Synthesis of Verified Dafny Methods. *Proceedings of the ACM on Software Engineering* 1, FSE (2024).

[29] Kedar S. Namjoshi. 2001. Certifying Model Checkers. In *CAV*.

[30] George C. Necula. 1997. Proof-Carrying Code. In *POPL*.

[31] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *PLDI*.

[32] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *CAV*.

[33] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *ICML*.

[34] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *TACAS*.

[35] Thomas Arthur Leck Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. In *PLDI*.

[36] Mary Sheeran, Satnam Singh, and Gunnar Stålmarck. 2000. Checking Safety Properties Using Induction and a SAT-Solver. In *FMCAD (LNCS, Vol. 1954)*. 108–125.

[37] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2024. Clover: Closed-Loop Verifiable Code Generation. *arXiv preprint arXiv:2310.17807* (2024).

[38] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. 2021. cake_lpr: Verified Propagation Redundancy Checking in CakeML. In *TACAS*.

[39] Ken Thompson. 1984. Reflections on Trusting Trust. *Commun. ACM* 27, 8 (1984), 761–763.

[40] Jean-Baptiste Tristan and Xavier Leroy. 2008. Formal Verification of Translation Validators: A Case Study on Instruction Scheduling Optimizations. In *POPL*.

[41] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *SAT*.

[42] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *ICLR*.

[43] Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Jianan Yao, Weidong Cui, Yeyun Gong, Chris Hawblitzel, Shuvendu Lahiri, Jacob R. Lorch, Shuai Lu, Fan Yang, Ziqiao Zhou, and Shan Lu. 2025. AutoVerus: Automated Proof Generation for Rust Code. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2, Article 396 (2025).

[44] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[45] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*.

[46] Emily Yu, Armin Biere, and Keijo Heljanko. 2021. Progress in Certifying Hardware Model Checking Results. In *CAV*.